U24AI006

ASSIGNMENT-13

Create a class for propositional symbols. For example: If you have a Symbol class for creating any propositional symbol, it should be written as:

p = Symbol('P')

Define unary and binary logic operations/logical connectives for handling simple propositional logic. (Define operations for not, or, and, conditional, and double conditional)

The inputs to your operations are all the possible values (boolean) of the constituent symbols and the outputs are also boolean values. Include another operation for printing a truth table for an operation that should mimic a truth table in the propositional logic as shown below:

P	**Q**	**P ∧ Q (P AND Q)**
F	F	F
F	T	F
T	F	F
T	T	T

Using the defined symbols and operations, find the truth values and print the truth tables of the following propositions: (write a function to evaluate complex propositions)

1. ~P->Q
2. ~P ∧ ~Q
3. ~P ∨ ~Q
4. ~P->~Q
5. ~P <-> ~Q
6. (P ∨ Q)∧(~P->Q)
7. ((P ∨ Q)->~R)
8. (((P ∨ Q)->~R) <-> ((~P∧~Q)->~R))
9. (((P->Q)∧(Q->R))->(Q->R))
10. (((P->(Q∨R)) -> (~P∧~Q∧~R)))

U24AI006

CODE(1):

```
from itertools import product

class Symbol:
    def __init__(self, name):
        self.name = name
        self.text = name

    def evaluate(self, values):
        return values[self.name]

    def variables(self):
        return [self.name]

class Formula:
    def __init__(self, text, func, names):
        self.text = text
        self.func = func
        self.names = names

    def evaluate(self, values):
        return self.func(values)

    def variables(self):
        return self.names[:]

def merge_names(left, right=None):
    names = []

    for name in left.variables():
        if name not in names:
            names.append(name)

    if right is not None:
        for name in right.variables():
            if name not in names:
                names.append(name)

    return names

def NOT(item, text=None):
    if text is None:
        text = "~" + item.text
    return Formula(text, lambda values: not item.evaluate(values),
merge_names(item))
```

```
def AND(left, right, text=None):
    if text is None:
        text = "(" + left.text + "/\\" + right.text + ")"
    return Formula(text, lambda values: left.evaluate(values) and
right.evaluate(values), merge_names(left, right))

def OR(left, right, text=None):
    if text is None:
        text = "(" + left.text + "\\/" + right.text + ")"
    return Formula(text, lambda values: left.evaluate(values) or
right.evaluate(values), merge_names(left, right))

def IMPLIES(left, right, text=None):
    if text is None:
        text = "(" + left.text + "->" + right.text + ")"
    return Formula(text, lambda values: (not left.evaluate(values)) or
right.evaluate(values), merge_names(left, right))

def IFF(left, right, text=None):
    if text is None:
        text = "(" + left.text + "<->" + right.text + ")"
    return Formula(text, lambda values: left.evaluate(values) ==
right.evaluate(values), merge_names(left, right))

def tf(value):
    if value:
        return "T"
    return "F"

def print_truth_table(number, formula):
    names = formula.variables()
    headers = names + [formula.text]
    widths = []

    for header in headers:
        widths.append(len(header))

    print("Proposition", number)
    print(formula.text)
    print(" | ".join(headers[i].center(widths[i]) for i in range(len(headers))))

    for row in product([False, True], repeat=len(names)):
        values = {}
        for i in range(len(names)):
```

```
            values[names[i]] = row[i]

        output = []
        for name in names:
            output.append(tf(values[name]))
        output.append(tf(formula.evaluate(values)))

        print(" | ".join(output[i].center(widths[i]) for i in range(len(output))))

    print()

def main():
    P = Symbol("P")
    Q = Symbol("Q")
    R = Symbol("R")

    not_p = NOT(P, "~P")
    not_q = NOT(Q, "~Q")
    not_r = NOT(R, "~R")

    propositions = []

    propositions.append(IMPLIES(not_p, Q, "~P->Q"))
    propositions.append(AND(not_p, not_q, "~P/\\~Q"))
    propositions.append(OR(not_p, not_q, "~P\\/~Q"))
    propositions.append(IMPLIES(not_p, Q, "~P->Q"))
    propositions.append(IFF(not_p, not_q, "~P<->~Q"))

    p_or_q = OR(P, Q, "P\\/Q")
    part6 = IMPLIES(not_p, Q, "~P->Q")
    propositions.append(AND(p_or_q, part6, "(P\\/Q)/\\(~P->Q)"))

    part7 = IMPLIES(p_or_q, not_r, "((P\\/Q)->~R)")
    propositions.append(part7)

    left8 = IMPLIES(p_or_q, not_r, "((P\\/Q)->~R)")
    right8_left = AND(not_p, not_q, "(~P/\\~Q)")
    right8 = IMPLIES(right8_left, not_r, "((~P/\\~Q)->~R)")
    propositions.append(IFF(left8, right8, "(((P\\/Q)->~R)<->((~P/\\~Q)->~R))"))

    p_to_q = IMPLIES(P, Q, "(P->Q)")
    q_to_r = IMPLIES(Q, R, "(Q->R)")
    left9 = AND(p_to_q, q_to_r, "((P->Q)/\\(Q->R))")
    propositions.append(IMPLIES(left9, q_to_r, "(((P->Q)/\\(Q->R))->(Q->R))"))
```

```
    q_or_r = OR(Q, R, "(Q\\/R)")
    p_to_q_or_r = IMPLIES(P, q_or_r, "(P->(Q\\/R))")
    all_not = AND(AND(not_p, not_q, "(~P/\\~Q)"), not_r, "(~P/\\~Q/\\~R)")
    propositions.append(IMPLIES(p_to_q_or_r, all_not,
"((P->(Q\\/R))->(~P/\\~Q/\\~R))"))

    print("Truth Tables")
    print()

    for i in range(len(propositions)):
        print_truth_table(i + 1, propositions[i])


if __name__ == "__main__":
    main()
```

OUTPUT:
Truth Tables

Proposition 1
~P->Q
P | Q | ~P->Q
F | F | F
F | T | T
T | F | T
T | T | T

Proposition 2
~P/\~Q
P | Q | ~P/\~Q
F | F | T
F | T | F
T | F | F
T | T | F

Proposition 3
~P\/~Q
P | Q | ~P\/~Q

F | F | T
F | T | T
T | F | T
T | T | F

Proposition 4
~P->Q
P | Q | ~P->Q
F | F | F
F | T | T
T | F | T
T | T | T

Proposition 5
~P<->~Q
P | Q | ~P<->~Q
F | F | T
F | T | F
T | F | F
T | T | T

Proposition 6
(P\/Q)/\(~P->Q)
P | Q | (P\/Q)/\(~P->Q)
F | F | F
F | T | T
T | F | T
T | T | T

Proposition 7
((P\/Q)->~R)
P | Q | R | ((P\/Q)->~R)
F | F | F | T
F | F | T | T
F | T | F | T

F | T | T | F
T | F | F | T
T | F | T | F
T | T | F | T
T | T | T | F

Proposition 8
(((P\/Q)->~R)<->((~P/\~Q)->~R))

P	Q	R	(((P\/Q)->~R)<->((~P/\~Q)->~R))
F	F	F	T
F	F	T	F
F	T	F	T
F	T	T	F
T	F	F	T
T	F	T	F
T	T	F	T
T	T	T	F

Proposition 9
(((P->Q)/\(Q->R))->(Q->R))

P	Q	R	(((P->Q)/\(Q->R))->(Q->R))
F	F	F	T
F	F	T	T
F	T	F	T
F	T	T	T
T	F	F	T
T	F	T	T
T	T	F	T
T	T	T	T

Proposition 10
((P->(Q\/R))->(~P/\~Q/\~R))

P	Q	R	((P->(Q\/R))->(~P/\~Q/\~R))
F	F	F	T
F	F	T	F

F	T	F	F
F	T	T	F
T	F	F	T
T	F	T	F
T	T	F	F
T	T	T	F